

DATA STRUCTURES LAB ASSIGNMENT 7

Q. Write a menu-driven C program to implement an Emergency Service Management System using an array-based Circular Queue. The system manages service requests received from customers. Each request contains the following information:

- Request ID
- Customer Name
- Service Type
- Priority
- Arrival Order

Priority levels:

- Priority 1 → Emergency
- Priority 2 → Urgent
- Priority 3 → Normal

Part A – Circular Queue Implementation

Implement a circular queue using an array of size N to store incoming service requests.

1. Insert a new request.
2. Delete the oldest request.
3. Peek at the oldest request.
4. Display all requests currently waiting.
5. Correctly handle FULL and EMPTY conditions.
6. Reuse positions freed after deletion.

Part B – Priority-Based Processing

When a service agent becomes available, the next request must be selected using the following rules:

7. Priority 1 requests are processed first.
8. Priority 2 requests are processed next.
9. Priority 3 requests are processed last.
10. If two requests have the same priority, the request that arrived earlier must be processed first.

Example:

Request	Priority
R101	3
R102	1
R103	2
R104	1
R105	3

The processing order should be:

R102 → R104 → R103 → R101 → R105

Part C – Menu-Driven Operations

1. Add New Request
2. Process Next Request
3. View Next Request
4. Display Waiting Requests
5. Display Requests by Priority
6. Display Total Waiting Requests
7. Exit

Implementation Constraints

- Use arrays only. Linked lists are not permitted.
- The circular queue must use front and rear indices.
- Circular indexing must be used when inserting and deleting requests.
- Do not sort the entire array every time a request is processed.
- For equal-priority requests, preserve FIFO order.
- Display suitable messages for queue overflow and underflow conditions.

Suggested Structure

Define a structure such as Request containing request ID, customer name, service type, priority, and an arrival sequence number. Maintain front, rear, and the current number of elements in the circular queue.